TOWARDS PARTICIPATORY CREATIVE SYSTEMS USING *MOBILE PROCESSING*

Christian Haines
Electronic Music Unit
Elder Conservatorium of Music
University of Adelaide
Adelaide, South Australia

ABSTRACT

In recent years interest has resurged in the field of creative networks and the participatory spaces they afford, especially in areas associated with computer music. Much of this interest has circled around small-scale networks consisting of cumbersome and specialised computer systems using sophisticated software. This investigation seeks to extend the scale of these networks, and hence participation, through the ubiquitous, intuitive and readily available technology of mobile phones. By extending the scale this author believes that unseen avenues of investigation will be revealed and new creative outcomes will occur.

1. INTRODUCTION

Historically the idea of a node, an interconnected point in space, represents a useful, although simplified, way to describe the players in creative participatory spaces. Classical musicians in an orchestra can be thought of as nodes, connected passively through a score and actively through a conductor and sound. An electro-acoustic ensemble may actively improvise around a set of principles or rules, whilst simultaneously having their contributions electronically manipulated by other players using varying degrees of active and passive engagement. Regardless of its simplicity, the concept of the node (or nodes) represents a suitable construct for the exploration of participatory creative structures moving beyond historical or normative models. Further, it enables players in these spaces to be considered 'unclassified,' where the notion of author shifts between audience, composer, performer and technology.

Nodal systems are an area of computer music that have undergone significant development and resurgent interest in the past decade or so. Drawing from the work of luminaries such as John Cage, with Imaginary Landscape #4, to David Tudor, and his various versions of Rainforest, researchers such as Gil Weinberg at the MIT Media Laboratory [21] and Atau Tanaka, Nao Tokui and Ali Momeni at the Sony Computer Science Laboratory in Paris [19], have sought to understand and chart nodal systems within participatory creative spaces. These systems include distributed and networked music, interconnected musical networks, interactive music systems and social music systems. Using this research as a point of reflection and drawing inspiration from the work of sound artists such as Mark Shepard and his Tactical Sound Garden Toolkit [14] and Scheimer and Havryliv's Pocket Gamelan [13], this investigation seeks to engage many of the same concepts and frameworks but shift them collectively to a larger, more volatile scale of investigation, embracing what Kim Cascone alludes to in the article, *The Aesthetics of Failure* [3].

2. PARTICIPATORY CREATIVE SYSTEMS

The investigation presented here is centred on the idea of developing a Participatory Creative System (PCS). A PCS can be thought of as large-scale creative works organised around a network of multiple nodes, where each node actively or passively contributes to the realisation of the work by permitting and/or promoting the interaction and exchange of ideas. A number of points will be drawn from a recently realised first incarnation of the work *[SOMETHING TO GO HEAR]* [5], a work for multiple mobile phones and software metronome which drew inspiration from the Fluxus-like work by György Ligeti *Poème Symphonique*[1] [10]. Essentially the work explores the notions of process music, distributed computing, spatialisation and technological determinism, with device variation yielding unforeseen sonic trajectories.

Essentially the work attempts to recreate aspects of the Ligeti piece. Audience members are provided with a software metronome, developed in Mobile Processing, for their mobile phone. The software on each phone produces a metronomic pulse with a differing tick, tempo and wind (potential energy that sets a duration limit on activity). At the beginning of the work audience members are asked to activate their software metronomes. Once activated each phone produces a short pulse of a sampled or MIDI-based metronome tick from its speaker. Over the course of the work a cluster of pulses is heard, with ticks phasing in and out of one another both temporally and spatially. Due to the varying tempos and winds the phones gradually stop at different times, revealing new sonic layers until all the phones have stopped.

[1] For 100 Metronomes.

2.1. Nodes

Each node in the network represents an individual or group of individuals contributing to the realisation of a creative work through means of a system. The individual is not necessarily artistically trained and may or may not have a clear sense of the overall direction or objective of the work. In essence, each node is a participant with their own specific function or role to perform, this role being determined both by the overall objective of the PCS and facilitated by the individual system used. Nodes may interact with one another passively, for example by simply performing together and having their musical or sonic results mixed acoustically. Subsequent variations and interactions can then be determined by the participants interpretation of that result. Finally, nodes may interact actively, for example in the case where the behaviour and interactions are a product of the sum of participant interactions and controls prior to acoustic propagation.

2.2. System levels

The system used in each node can vary significantly depending on the PCS but typically operates at two levels. The first of these is the node level system, a technological system designed to facilitate and encourage individual or group participation in the work; the second is the network level system which can either be considered as the sum of the node level systems, or as node level systems conducted and/or directed by other nodes.

2.3. Participation

Each aspect of the system makes use of a technology, combining software and hardware, whose inherent qualities and explicit design promote participation. Participants are empowered to contribute to the realisation of a creative work because:

(a) they have ready access to the hardware required to contribute;

(b) the software is provided for free, adheres to principles of perceptive interaction design and encourages play;

(c) system interaction at global network level is intuitive in terms of its behaviour.

2.4. Objectives

The key objective in developing a PCS is to foster the possibilities that participatory networks open up, the questions they raise and the inherent potential of the system. This includes the possibility of revealing pathways for new creative interpretation or activity through large-scale clustering of nodes (participants). In order to achieve this key objective, participation must be:

(a) (easily) *facilitated*, making use of readily available hardware and software, including intuitive interactions and providing explicit causality within the networks;

(b) *transient*, allowing participants to engage as part of a work but disengage readily without affecting the sum of the work;

(c) *scalable* (within reason), enabling the number of participants to reduce or increase whilst still producing a cohesive and relevant creative result;

(d) *socialised*, as discussed in *Facilitating collective musical creativity* [19], moving beyond the notion of participants as flaccid nodes, utilitarian in their role in providing technology in a work, to the idea that they themselves validly contribute and engage with other participants meaningfully;

(e) of a *high magnitude*, with network participation, and hence the amount of nodes, at a mass critical enough to begin to draw a sense of the potential of the work and its ability to raise new questions.

3. MOBILE PHONE AS AGENT

As can be noted with other technological platforms, a mobile phone[1] is a combination of hardware, operating system and software. The combination of these elements, coupled with the device's interaction design, determines the inherent capability and value of the device in terms of its creative agency. Creative agency in this sense relates to the kinds of creative outputs that can be produced using the mobile phone and the inherent participatory pull or allure that such devices have in relation to the socialised aspect of the work i.e. facilitating participation. In the case of *[SOMETHING TO GO HEAR]* participation was easily facilitated through hardware profiling, Internet and point-to-point network distribution. Further, users were able to readily test the software through a simple and direct user interface.

3.1. Hardware

In considering the hardware, a mobile phone is a ubiquitous[2] mobile computing device. As with other mobile computing devices, the hardware is based upon an integrated and cooperative array of components including a CPU, permanent memory, RAM, standard

[1] Although a mobile phone represents what some may consider a fairly standard profile of technologies, it is worth noting that such profiles vary enough in the case of this examination to be deemed very different technologies. As such the mobile phones discussed in this paper are those that utilise the Java Virtual Machine (JM) and at least in this sense provide a degree of cross-platform operability. Higher profile phones, such as Windows Smart Phones, Apple's iPhone and Google's Android were not considered.

[2] Ubiquitous in this sense refers to the idea the phone is readily available, common technology, used in abundance in modern industrialised societies.

inputs (keypad, buttons, microphone, Bluetooth, radio telephony and WiFi*)* and standard outputs (display, speaker, Bluetooth, phone signal and WiFi). As with other devices, the hardware itself places a number of restrictions on the kinds of creative works that can be produced. Specifically, mobile phones fit into the category of devices known as resource constrained.

3.1.1. Processor and memory

Of particular note in relation to the impact of hardware on the creative agency are the limitations on the kinds of processes that the CPU can deal with at a time and what is able to be stored in memory for such processes. Standard generation phones (i.e. not smart phones) vary in the speed from a few dozen to several hundred megahertz. Further, the kinds of instruction cycles that are able to be executed will likewise vary. Suffice to say the inherent processor strength of mobile phones is modest compared to PDAs, laptops and desktop computers.

3.1.2. Speaker

In terms of sound output, mobile phones usually provide options such as ear speakers, loud speakers, wired and Bluetooth headsets. In respect to frequency range and dynamics, headset options usually provide a better quality signal than a speaker. Speakers typically suffer as a result of the phone's physical structure, and hence acoustics, as well as the speaker size and design. Of paramount concern for some manufacturers is the cost-benefit trade-off of having recognisable 'voice' versus speaker price, thus usually resulting in low fidelity speakers. This said, in terms of performance logistics and simplifying participation, the speaker phone is the most suitable option as it provides a convenient and simple method of sound propagation.

3.2. Operating system

As with other computing devices, the operating system of mobile phones varies considerably between manufacturers and is usually proprietary. For example, Sony-Ericsson, Nokia, Samsung and Motorola all provide software development kits (SDKs) for use in developing on their mobile phone operating systems. In an effort to better support and encourage software development, particularly across platforms, many manufacturers have chosen to support the Sun's Java Platform 2 Micro Edition (J2ME)[15] in their operating systems.

J2ME "provides a robust, flexible environment for applications running on mobile and other embedded devices…. Applications based on Java ME (J2ME) are portable across many devices, yet leverage each device's native capabilities." [15]. J2ME runs within the Java Virtual Machine (VM) that sits on top of the mobile phone's operating system. Although this virtualisation simplifies many of the tasks for development, it introduces a notable performance sacrifice with respect to processing cycles [6]. The configuration of devices and compliance with respect to J2ME is determined by the Connected Limited Device Configuration (CLDC)[18], which determines the framework by which low level programming interfaces can interact with the phone's hardware and operating system. On top of this, and complicating matters further, is the Mobile Information Device Profile (MIDP)[16], a specification that determines a range of APIs to access the phones hardware and OS feature set. Simply put J2ME provides a cross platform means to develop software for resource constrained devices, such as mobile phones, but paradoxically does so in a manner that constrains those resources further.

3.3. Programming environment

Since circa 2000, for the purposes of creativity, there has been a proliferation in the number of programming environments that seek to provide higher levels of abstraction to low level features of devices. One environment that has received significant recognition, and a growing user base, is Ben Fry's and Casey Rea's *Processing* [4]. *Processing* "is an open source programming language and environment for people who want to program images, animation, and interactions" [4]. Building upon the foundation of *Processing* is Francis Li's *Mobile Processing* [7]. *Mobile Processing (*MP) seeks to extend the objectives of *Processing* to Java powered mobile phones. This is undertaken by providing a simple, intuitive environment through which mobile phone applications can be authored, tested and deployed. Although MP does open up the mobile phone for artistic exploration, it contains a reduced feature set of the environment *Processing*. Further, and similar to *Processing*, it has limited features in the areas of event timing, MIDI and sound input/output/playback. Some of these limitations have been overcome through additional base and third-party libraries, particularly those developed by Mary Jane Soft [11], which has spent considerable energy providing MP with access to new functions and processes in J2ME, such as extended MIDI, sound and communications.

3.4. System variability

Although MP provides a pathway for artists wishing to develop mobile phone applications without requiring specialised knowledge of programming, its stability is undermined by cross-platform compatibility and language abstraction levels. Stability can be defined as the consistent and uniform behaviour of a device in relation to its set tasks, regardless of variability. Variability describes the disparity that results from the interconnectedness between differing hardware profiles, the components that make up a hardware profile, phone carrier operability, operating system implementation of virtualisation, compliance and implementation of the virtualisation, and the programming environment's abstraction of the language.

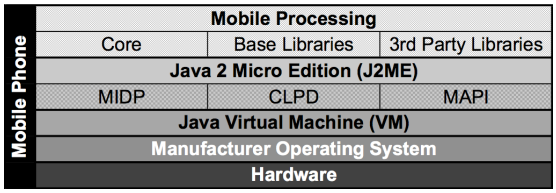


Image 1 - Mobile Phone System Layers

Given the number of layers, development can be wrought with difficulties, particularly in respect to producing consistent results. In this regard, developing multiple safe guards is a must to ensure processes can be *tried* and problems caught. Regardless, there have been instances where *trying* everything means *catching* everything, and no processes have been left functioning.

Obviously a choice could be made to directly engage with the phone and its system through manufacturer specific SDKs, or even by using J2ME directly. However, it is clear there is a learning curve to be addressed before any meaningful artistic product can be produced, and this may effect artistic participation. This raises a point of tension between the need-to-know, the need-to-do and the balance between flexibility, ease of use and whether or not to treat unintended consequences as issues or idiosyncratic and endearing personality traits presented by the technology. Further, even models from the same generational family or of exactly the same make have shown inconsistent behaviour.

For example, during *[SOMETHING TO GO HEAR]* two phones of exactly the same manufacturer and model (Nokia 6120) produced longer movements (ensuing in a duel lasting longer than the other fourteen participants) and one sounded a completely unanticipated rapid staccato of notes close to finishing. The timing of the movement was fixed, leading to the only explanation for this behaviour being that the phone had some kind of inherent problem.

4. ISSUES IN IMPLEMENTATION

Typically, each creative programming environment aims to serve either a particular niche or a broad set of creative objectives. In many instances where the prescribed range of objectives is narrow, for example visually-centric, there are often individuals who are keen to extend the functionality to other realms, for example music and sound. *Processing*, whose objectives are stated above, is situated in a visual context and has been extended through third-party libraries. Similarly, MP has undergone such an extension. However, regardless of the apparent evolution of the environment, some fundamental constraints built into the gene pool present ongoing challenges for those desiring to work with sound and/or music.

4.1. Frame based timing

The most notable issue with MP is that of timing. MP inherited a frame-based timing mechanism from *Processing*. In short, divisions of time are determined by the frame rate (FR) i.e. a number of frames per second (FPS). The FR determines the speed at which events are produced in the environment and the speed that the particular device is able to handle adequately.

4.1.1. Direct frame rate to BPM conversion

A direct frame to tempo conversion yields a basic range of tempos to work with. For example, a FR of 25 FPS can, in theory, produce 1500 frames or pulses or beats per minute. Therefore, shifting the FR up and down will produce different and somewhat awkwardly spaced, tempos (Table 1).

Frame Rate	30	25	20	15	10	5	4	3	2	1
BPM	1800	1500	1200	900	600	300	240	180	120	60

Table 1 –Frame Rate to BPM Conversion

4.1.2. Skip frames to diversify tempo

As has been discussed elsewhere by those who develop music applications using frame-centric technology (including Adobe's Flash and Director)[1], getting finer increments of tempo requires rethinking the paradigm of timing in terms of division of frames. This idea can be achieved by using Frame Skip (FS), which skips over a number of frames at a fixed FR before issuing a beat or pulse, and this provides finer increments (Table 2).

Frame Rate 25											
SKIP	30	25	20	15	14	13	12	11	10	9	8
BPM	50	60	75	100	107	115	125	136	150	167	188

Table 2 - Frame Rate and Skip to BPM Conversion

4.1.3. Frame-to-Tempo lookup table

A finer degree of tempo resolution can be further achieved by simultaneously varying both the FR and FS. A pre-calculated lookup table, indexed by FR and FS, provides the best apparent degree of tempo control within this restricted visual paradigm. Having said that, the implementation of this as a real-time control in *[SOMETHING TO GO HEAR]* introduced noticeable timing artefacts, such as the tempo never accurately resolving to the newly selected tempo during run-time operation. As a result, the method was withdrawn from use until further development could resolve the issue (Table 3).

		FPS					
		30	25	20	15	10	5
SKIP	50	30.0	24.0	18.0	12.0	6.0	4.8
	40	37.5	30.0	22.5	15.0	7.5	6.0
	30	50.0	40.0	30.0	20.0	10.0	8.0
	20	75.0	60.0	45.0	30.0	15.0	12.0
	15	100.0	80.0	60.0	40.0	20.0	16.0
	10	150.0	120.0	90.0	60.0	30.0	24.0
	5	300.0	240.0	180.0	120.0	60.0	48.0

Table 3 – Frame and Skip BPM Lookup Table

4.1.4. *Thread sleep alternatives*

Given that MP controls the FR via a system or thread sleep, and can in fact make use of this thread sleep for finer timing control, MP could become more attractive for those requiring precise timing control. As has become a familiar story, there are trade-offs in providing highly abstracted languages that promote ease of use, but reduce the level of control by abstracting to functions and classes. An example of this is embedding thread sleeps under the umbrella of frame rate control. Further, the degree of thread sleep is finite, given processor cycles require a certain period of time in which to operate.

4.1.5. *Error offset in timing and absolute time*

Although the above methods of tempo control at least appear to provide a stable basis by which events can be issued, even if increments are restricted, there is an error offset between those events. This varies as a ratio and is dependent upon the method chosen, such as FR to BPM conversion, FR with FS to BPM conversion and variable FR with variable FS to BPM conversion. For example, on a Nokia 6288 at a FR of 25, beat divisions should be 40 milliseconds but in actuality they vary from 36 – 44, representing an error of between 0 and 10%. The main issue resulting from this relates to the introduction of a 'swing feel' to the timing, which at certain FR and FS combinations becomes heightened and almost unpleasant.

Nokia 6288 with timing sketch (no screen drawing)								
Beat	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Interval (ms)	43	38	37	41	40	44	36	42
Offset (ms)	3	-2	-3	1	0	4	-4	2
Error (%)	7.5	5.0	7.5	2.5	0.0	10.0	10.0	5.0
Frame Rate: 25	**Frame Skip: None**			**BPM: 1500**		**Interval (ms): 40**		

Table 4 - Frame Rate Timing Errors

4.1.6. *Interaction with display elements*

As mentioned previously, the apparent sluggishness of the VM means that switching tempos dynamically mid-stream is not an elegant proposition. Further, and importantly, MP can suffer severe 'rate pull down' when utilising certain classes and functions, particularly those that generate display output. For example, on a Nokia 6288 the frame rate can operate optimally at 20 FPS or a pulse every 50 milliseconds. However, once user interface elements are included as part of the onscreen render, this degrades down to around 80 milliseconds or 12 FPS. This is a notable and unacceptable deterioration in performance and, in the case of using a tempo look-up table as discussed above, restricts the timing resolutions that can be used.

4.1.7. *System variability and frame rates*

Finally, as mentioned before, the makeup of the device and its variability dictates how efficiently and consistently tasks are handled. The issue here in terms of promoting a socialised use of technology, and hence participation, is that a device's tempo range and stability will differ significantly between participants, thereby typically producing nodes that are ineffective or simply non-cohesive. Although such consequences are unintended, there maybe instances, as mentioned before, were the results are desirable and not unwelcome.

4.2. Sound

4.2.1. *General capabilities*

The base level sound capabilities of MP are restricted, and are provided through a library. In terms of MIDI, only basic tone playback is provided with pitch and volume selection. Notably, the default tone that is selected for MIDI playback varies in type and quality between devices. Further, attempting to instantiate multiple voices can produce memory errors and/or device crashes. With regard to sound file playback, only certain audio file types are supported with basic volume control, and audio capabilities are clouded by the fact that devices vary in the file types they support. As with MIDI, multiple instantiations of audio file playback can also produce memory errors and device crashes. For example, in developing the software for *[SOMETHING TO GO HERE]*, two systems for metronome playback were developed (one MIDI-based and one audio-file based) in an attempt to provide a safe guard option if the mechanism did not work. In one instance, with a Nokia E51, neither option functioned, although the MP phone profiler [9] indicated both methods were supported.

4.2.2. *Extensions*

Mary Jane Soft's MSound, MSynth and MAudio3D libraries [11] provide welcome extensions to the base sound capabilities of MP and build upon the J2ME technologies tied with the MIDP 2.0 specification [17]. These provide access to MMAPI 1.1 (Mobile Media API) [16] and a number of functions including: mixing sound streams; performing audio capture and playback; a relatively extensive range of PCM and CODEC audio file types; simulation of 3D sound; basic sound generation and synthesis; and extended MIDI functionality including instrument selection and channel use. Again it can be noted that these libraries extend the range of layers above the system and could, in turn, produce their own idiosyncrasies within the system. For example, MIDI program change initialisation failure has required the rather ad hoc solution of loading a MIDI file and playing it back briefly.

4.3. Other issues

A number of other issues have arisen during the course of exploring the viability of MP and mobile phone technology as a socialised participatory device.

4.3.1. Memory handling

As mentioned previously, some of the memory handling with respect to voice creation and stopping, i.e. event timing or instrument usage, is problematic within certain families of phones such as the Nokia 6200 series and the Sony Ericsson K series. In the case of preparing *[SOMETHING TO GO HEAR]* this was notable and represented a fundamental reason for different sound generation systems being introduced.

4.3.2. Array executions

In some cases array instantiation produces what the system deems as malformed arrays, often leading to 'out of bound' errors on particular devices, whilst other devices perform the same function without issue. This is highly suspect given this issue can often occur upon initialising an application, which then proceeds to operate correctly after the next start-up. In the performance of *[SOMETHING TO GO HERE]* this required some participants to do a 'warm start' by starting their device and permitting the crash to happen, thereby allowing it to function correctly for the performance.

5. ADVANTAGES IN IMPLEMENTATION

5.1. Universal

Despite many of the reservations expressed, MP's implementation of J2ME provides a relatively universal platform to give programmers access to different devices. Most reservations concerning inconsistencies lie with device manufacturers and their implementation with respect to hardware. This is a key consideration in terms of facilitating the development of participatory systems, and perhaps allowing participation to extend to the area of application building or modification. This could take place, for example, by providing PCS frameworks that can be modified, changed and redeployed by participants or those wishing to implement their own systems.

5.2. Ubiquitous

The ubiquity of mobile phone / computing technology, coupled with platform universalisation, provides a relatively solid foundation by which applications can be optimistically deployed and run on a large scale. Although based on the experience of developing and realising *[SOMETHING TO GO HEAR]*, it is clear that one should proceed with caution and aim to provide a number of safe guards, such as alternate versions or capabilities. Similarly one should be prepared for some systems not to participate and gauge the effect this has on the overall realisation of a work.

5.3. MIDI and sound functionality

Although primitive, the inclusion of MIDI and sound functionality does provide a basis by which sonic or musical outcomes can be produced and, as mentioned, Mary Jane Soft's libraries have taken these functions further since 2007, providing avenues for exploration. The functionality available in mobile systems can be likened to looking at modern technology through a lens of times past. Although we see a modern communications device, the sound and music capabilities are seemingly as primitive as those seen and used by founders such as Tudor, Cage, Dodge and Lansky et al, between the 1960s and 1980s. Regardless, a PCS seeks to leverage such basic capabilities through the clustering of devices and revealing pathways of new creative interpretation or activity, rather than seeking totalised outcomes through an extensive and complex array of capabilities in one device.

5.4. Communications systems

One of the main attractions with mobile phones as a nodal participatory technology lies in its ability to communicate in a multitude of ways, such as radio telephony, short message service (SMS), multimedia messaging service (MMS), Bluetooth, WiFi and GPRS. These methods provide different mechanisms by which information can be transmitted, whether these mechanisms are: short range point-to-point, such as Bluetooth which is useful for rapid connectivity and low data exchanges, as seen with games; broader exchanges through intranets or the internet where centralised repositories of information or distant geographical interactions are required; radio telephony allowing for the feed of voice or other acoustic spaces to another location; the highly socialised use of SMS which has been exploited in range of mobile phone based artworks; and GPRS, which provides localised information and positioning. In short, the range of communication options available presents a pivotal advantage by promoting exchange or transmission of content. Further, due to participants typically being familiar with these systems, it provides an obvious means by which creative applications can encourage participation. Finally, the breadth of simple information types, such as voice/audio, text, photo and video, that can be collected and transmitted easily without training in new systems provides a range of possibilities for different creative works.

6. FUTURE DIRECTIONS

Although this examination of mobile phones, as a central part of participatory creative systems, is still in its infancy, it is clear from the explorations in *[SOMETHING TO GO HEAR]* that the majority of effort has been expended mediating the vast variety of devices. Bearing this in mind, the following core objectives have been identified:

(a) Reducing support by developing systems that provide safe guard positions which are intuitive, graceful and provide feedback to the participant. This will no doubt include profiling system functionality on a more detailed level and collating

the information in a central repository, similar perhaps to the phone profiling used by Francis Li [9].

(b) Extending multi-system sound playback. As the development of PCS is situated from a context of working with sound, this objective is obvious and essential.

(c) Extending communicative interoperability between devices and beyond the socialised and acoustic realms, allowing the network to perform more like a traditional network and leveraging interactions similar to those discussed by Weinberg [21].

7. CONCLUSION

In summary, a participatory creative system seeks to build on and contribute to the field of network music and interactive music systems by extending the scale and level of participation within a localised and socialised space. By extending the scale it is believed that new creative opportunities and questions will be revealed. Further, by extending the scale it is duly noted that trade-offs have been made in terms of what kinds of systems can facilitate participation and also actively contribute meaningful creative capital to the outcome.

The mobile phone can be seen as a suitable candidate and facilitator for a PCS. However, the mobile phone does provide a point of tension by advancing a ubiquitous platform that promotes contributions from participants with a device that is inherently variable and reduced in capability when compared to other modern technologies. Rather than seeking absolute resolution of issues associated with variability, within reason, this investigation embraces variability as tenet of technological determinism and looks to the personality and potential that this variability can bring to the outcomes and social space of the participatory creative system.

8. REFERENCES

[1] Adobe. "Adobe". 2009. 20/4 2009. <http://www.adobe.com/>.

[2] ARTE. György Ligeti - Poème Symphonique for 100 Metronomes. Video. ARTE (France).

[3] Cascone, Kim. "The Aesthetics of Failure: “Post-Digital” Tendencies in Contemporary Computer Music." Computer Music Journal 24.4 (2000): 12-18.

[4] Fry, Ben and Reas, Casey. Processing. Vers. 1.03. Computer software, 2009.

[5] Haines, Christian. [Something to Go Hear] Rec 16/4/2009. Audio, Adelaide, 2008.

[6] Java User Group Community. "Tastephone Server". 2009. 20/4 2009. <http://www.club-java.com/TastePhone/J2ME/MIDP_mobile.jsp>.

[7] Li, Francis. Mobile Processing. Vers. 0007. Computer software, 2004.

[8] Li, Francis. "Mobile Processing Reference". 2009. 20/2 2009. <http://mobile.processing.org/reference/>.

[9] Li, Francis. "Phones". 2009. Profiling area for mobile phone compatibility with Mobile Processing. 20/3 2009. <http://mobile.processing.org/phones/index.php>.

[10] Ligeti, György, and Pierre Charial. Mechanical Music. sound recording. Sony Classical, New York, 1997.

[11] Manrique, Marlon.J. Mobile Processing Libraries. Mary Jane Soft 2009.

[12] Nokia. "Nokia 6288". 2009. Nokia. 14/9 2008. <http://www.nokia.com.au/A4519121>.

[13] Scheimer, Greg and Havryliv, Mark. "Pocket Gamelan: An Extensible Set of Microtonal Instruments." Australasian Computer Music Conference - Generate + Test. ACMA.

[14] Shepard, Mark. "Tactical Sound Garden". 2008. 17/11 2008. <http://www.tacticalsoundgarden.net/>.

[15] Sun Microsystems. "Java Me at a Glance". 2009. Sun. 20/4 2009. <http://java.sun.com/javame/index.jsp>.

[16] ---. "Mobile Media API". 2009. Sun. 17/1 2009. <http://jcp.org/en/jsr/detail?id=135>.

[17] ---. "Mobile Information Device Profile 2.0". 2009. Sun. 17/1 2009. <http://jcp.org/aboutJava/communityprocess/final/jsr118/index.html>.

[18] ---. "Connected Limited Device Configuration 1.1". 2009. Sun. 16/1 2009. <http://jcp.org/aboutJava/communityprocess/final/jsr139/index.html>.

[19] Tanaka, Atau, Nao Tokui, and Ali Momeni. "Facilitating Collective Musical Creativity." Proceedings of the 13th annual ACM international conference on Multimedia. ACM.

[20] Tudor, David. Rainforest (Version 1). Compact Disc. Ellipsis Arts, 1990.

[21] Weinberg, Gil. "The Aesthetics, History, and Future Challenges of Interconnected Music Networks." International Computer Music Conference. ICMA.